

LECTURE 6

Integrity Constraints

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

Agenda

- Domain constraints — keeping individual values legal
- Entity integrity — why no primary key component may ever be NULL
- Referential integrity — keeping foreign keys honest
- General constraints — encoding business rules the model doesn't know about
- How the four constraint categories fit together
- Enforcing referential integrity: CASCADE, RESTRICT, SET NULL

Why Constraints Matter

- Lecture 5's schema says what shape data must take, but says nothing about whether a particular row makes sense.
- Nothing in that schema stops a staff member with no ID, a salary of –5,000, or a property assigned to a branch that does not exist.
- Integrity constraints close that gap, and the DBMS enforces them automatically.
- A query result is only trustworthy if the DBMS refused every write that would have made it false.

The Extended Schema

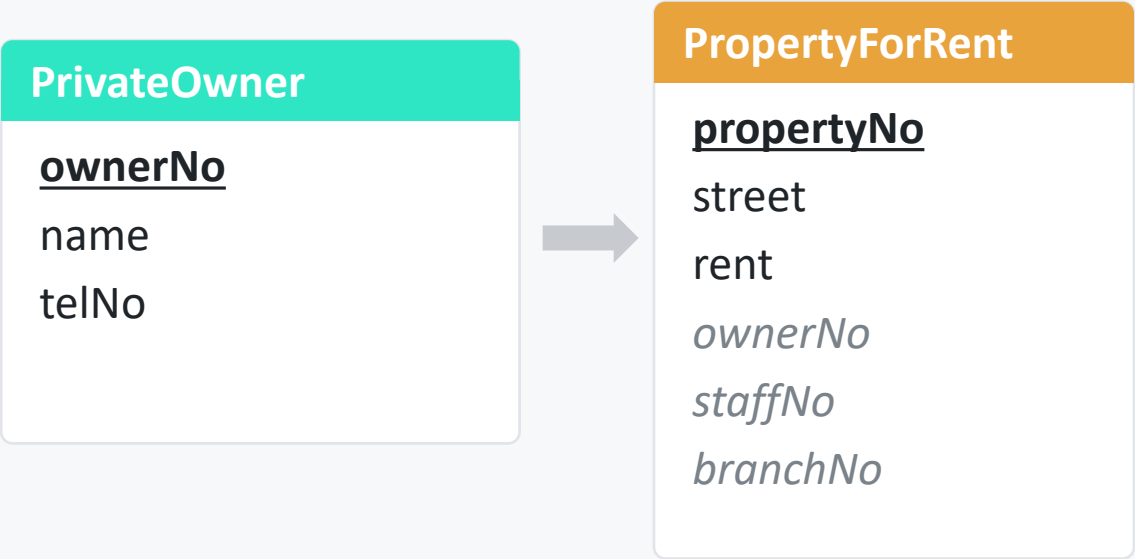
PropertyForRent carries three foreign keys at once — ownerNo, staffNo, branchNo — the perfect relation for testing every integrity rule.

PrivateOwner (ownerNo, name, telNo)

ownerNo	name	telNo
CO46	Joe Keogh	021-234-5678
CO87	Carol Farrel	021-556-7890
CO40	Tina Murphy	042-234-1122
CO93	Tony Shaw	042-556-9988

PropertyForRent — 9 attributes, 3 foreign keys

propertyNo	type	rent	ownerNo	staffNo	branchNo
PA14	House	22000	CO46	SG14	B003
PL94	Flat	15000	CO87	SA9	B007
PG4	House	35000	CO40	SG14	B003
PG36	Flat	24500	CO93	SG37	B003
PG16	House	45000	CO93	SA9	B007
PG21	Flat	28000	CO87	SL21	B005



Worked Example: Reading the Foreign Keys

PropertyForRent carries three foreign keys at once

Relation	Foreign key	References	Why it exists
PropertyForRent	ownerNo	PrivateOwner(ownerNo)	Which owner lists the property
PropertyForRent	staffNo	Staff(staffNo)	Which staff member manages it
PropertyForRent	branchNo	Branch(branchNo)	Which branch it belongs to

- Step 1 — find the primary key: propertyNo (underlined in the schema).
- Step 2 — find each foreign key: the three attributes that reproduce another relation's key.
- Step 3 — check which of them may be NULL: staffNo may be NULL (an unassigned property); branchNo may not.

Domain Constraints

A domain constraint restricts a single attribute's legal values to its declared domain — the most basic integrity rule, checked in isolation from any other row or table.

```
CREATE TABLE PropertyForRent (  
    propertyNo  VARCHAR(5)    NOT NULL,  
    rooms       SMALLINT      NOT NULL CHECK (rooms > 0),  
    rent        DECIMAL(8,2)  NOT NULL CHECK (rent > 0),  
    type        VARCHAR(5)    NOT NULL CHECK (type IN ('House', 'Flat')),  
    ...  
);
```

Domain constraints vs. general constraints

“Rent must be positive” is a domain constraint — one value, one rule. “Rent must be at least 80% of the average rent for that property's city” compares against other rows — that graduates to a general constraint.

Worked Example: Testing Domain Constraints

Attempted row	Constraint that applies	Result
rooms = -2	CHECK (rooms > 0)	Rejected
type = 'Cabin'	CHECK (type IN ('House', 'Flat'))	Rejected
rent = 22000	CHECK (rent > 0)	Accepted

- Each row is tested against one attribute's domain in isolation — no other row is consulted.
- A value that fails any CHECK is refused before it is stored, so the bad value never exists in the table.

Entity Integrity

No attribute participating in a relation's primary key may hold a NULL value.

A primary key's entire job is unique identification. NULL means “unknown” — a row whose identifying value is itself unknown can't reliably be told apart from another row that is also missing it.

Attempted insert into Staff — REJECTED

staffNo	name	position	salary	branchNo
(NULL)	Zara Malik	Assistant	11000	B005

staffNo declared PRIMARY KEY implies NOT NULL automatically — UNIQUE alone still permits NULL.

Composite keys: EVERY component must be non-NULL

In Viewing(clientNo, propertyNo, viewDate, comment), a tuple with a known clientNo but a NULL propertyNo still violates entity integrity — the pair identifies the tuple, and one missing half is enough to break that.

Non-key attributes can still be NULL

Staff.position or PropertyForRent.rooms may legitimately be NULL — the rule is narrowly about the primary key.

Worked Example: NULL in a Primary Key

clientNo	propertyNo	viewDate	comment	Result
CR56	PG4	2026-06-01	too remote	Accepted
CR56	NULL	2026-06-15	NULL	Rejected — key part is NULL
CR62	PA14	2026-05-26	NULL	Accepted — NULL comment is fine

- Viewing(clientNo, propertyNo) is a composite key: every component must be non-NULL, not just one of them.
- Only key attributes are covered by entity integrity; a NULL in a non-key attribute such as comment is legal.

Referential Integrity

If a foreign key exists, its value must (1) match a candidate key value currently in the referenced relation, or (2) be wholly NULL.

Attempted insert — REJECTED (SX99 doesn't exist)

propertyNo	ownerNo	staffNo	branchNo
PG55	CO46	SX99	B003

Accepted insert — foreign key wholly NULL

propertyNo	ownerNo	staffNo	branchNo
PG55	CO46	(NULL)	B003

staffNo = NULL doesn't claim any relationship at all — nothing to be inconsistent with. A property may have no staff assigned yet, but never a fabricated one.

Composite foreign keys: wholly NULL, or not at all A foreign key NULL in one column but real in another is itself a violation.

Self-referencing foreign keys

A legal special case: Branch(branchNo, street, city, postcode, mgrStaffNo) could hold mgrStaffNo as a foreign key into Staff — and Staff already references Branch. A relation can even reference itself, e.g. Staff.supervisorStaffNo → Staff.staffNo.

Recap: Which Side Changed?

- Insert into PropertyForRent with staffNo = SX99 → referencing side, no such staff member → always rejected.
- Insert with staffNo = NULL → legal, because a wholly NULL foreign key claims no relationship at all.
- Delete staff member SG14 who manages PA14 and PG4 → referenced side → the policy you declared (RESTRICT, CASCADE, or SET NULL) decides the outcome.

General Constraints

Organization-specific business rules that go beyond domain, entity, and referential integrity — the relational model has no built-in vocabulary for them.

- No member of staff can manage more than 100 properties at a time.
- A manager's salary must exceed every assistant's salary at the same branch.
- A property's rent must fall between PKR 5,000 and PKR 100,000.
- Staff can't handle a viewing for a property they themselves manage.

```
ALTER TABLE PropertyForRent
  ADD CONSTRAINT chk_rent_range
  CHECK (rent BETWEEN 5000 AND
100000);
```

Simple, single-table CHECKs are easy for the DBMS to enforce directly. Rules comparing across many rows (“manager > every assistant”) usually need triggers, stored procedures, or application logic — but they remain integrity constraints either way.

Worked Example: Checking a Business Rule

Proposed rent	CHECK (rent BETWEEN 5000 AND 100000)	Result
45000	$5000 \leq 45000 \leq 100000$ ✓	Accepted
150000	$150000 > 100000$ ✗	Rejected
3000	$3000 < 5000$ ✗	Rejected

- A single-table CHECK like this one is easy for the DBMS to enforce directly.
- Rules that compare across rows (a manager's salary above every assistant's at the same branch) need triggers or application logic.

Integrity Constraints, Together

Domain constraint

One attribute's value against its declared domain — e.g. $\text{rent} > 0$



Entity integrity

No primary key component may be NULL



Referential integrity

Every non-NULL foreign key value must match an existing referenced key



General constraint

Organization-specific business rule, spanning any number of attributes or rows

All four exist for the same reason: a database is only trustworthy if the DBMS refused every write that would have made a query result false.

From Four Constraints to Enforcement

- Domain, entity, referential, and general constraints all describe the same goal: what is stored must reflect reality.
- The DBMS enforces them on every INSERT and UPDATE — your application code does not have to remember to check.
- Domain and entity violations are always rejected outright; referential violations depend on which side of the relationship changed.

Enforcing Referential Integrity

Domain and entity integrity: the offending INSERT/UPDATE is simply rejected. Referential integrity is more interesting — a violation can come from either side, so the DBMS needs a policy per direction.

Deleting Staff row SG14 (who manages PA14 and PG4) — three referential actions:

RESTRICT

Refuse the delete while
PA14/PG4 still reference SG14



CASCADE

Delete SG14, then auto-delete
PA14 and PG4 too



SET NULL

Delete SG14, set staffNo to NULL
on PA14 and PG4

```
CREATE TABLE PropertyForRent (  
    ...  
    staffNo    VARCHAR(5),  
    branchNo   VARCHAR(4) NOT NULL,  
    FOREIGN KEY (staffNo) REFERENCES Staff(staffNo) ON DELETE SET NULL,  
    FOREIGN KEY (branchNo) REFERENCES Branch(branchNo) ON DELETE RESTRICT  
);
```


Step by Step: Tracing a DELETE on Staff

Declared action	What happens to PA14 and PG4	Outcome of DELETE SG14
RESTRICT	Still reference SG14 → nothing changes	Refused with an error
CASCADE	Both rows are deleted as well	Staff row and both properties gone
SET NULL	Both rows kept, staffNo set to NULL	Staff row gone, properties unassigned

- Step 1 — find the referencing rows: PA14 and PG4 both have staffNo = SG14.
- Step 2 — apply the declared policy to those rows, then remove SG14.

Choosing a Referential Action

This is a design decision, not a default Ask: “if the referenced row disappears, does the referencing row still make sense on its own?”

Relationship	If the referenced row disappears...	Favor
Viewing → Client	A viewing record makes no sense without the client	CASCADE / RESTRICT
PropertyForRent → staffNo	The property still makes sense, just unassigned	SET NULL
PropertyForRent → branchNo	A property cannot exist without belonging to some branch	RESTRICT

Getting this wrong is a common, expensive real-world bug — either silent data loss from an over-eager CASCADE, or a wall of RESTRICT errors when SET NULL would have been sensible.

Worked Example: Choosing a Referential Action

Relationship	Does the child still make sense without its parent?	Choose
Viewing → Client	No — a viewing needs the client who made it	CASCADE or RESTRICT
PropertyForRent → Staff	Yes — a property can wait for a new manager	SET NULL
PropertyForRent → Branch	No — a property belongs to a branch (NOT NULL)	RESTRICT

- Ask: if the referenced row disappears, does the referencing row still make sense on its own?

Try It Yourself

- PrivateOwner CO46 is deleted and owns PA14 and PG4. Which referential action on PropertyForRent.ownerNo would you declare, and why?
- Write the CHECK constraint that keeps rooms positive and limits type to House or Flat, as it would appear in a CREATE TABLE statement.
- Could a Viewing row with clientNo = NULL ever be legal? Justify your answer using entity integrity.
- Give one general constraint for the agency that cannot be expressed as a simple domain check, and say how you would enforce it.

KEY TAKEAWAYS

Integrity Constraints

- Domain constraints restrict a single attribute's value to its declared, legal domain.
- Entity integrity: no attribute that is part of a primary key — single or composite — may ever be NULL.
- Referential integrity: every non-NULL foreign key value must match an existing candidate key value, or be wholly NULL.
- General constraints capture business rules the relational model has no built-in vocabulary for.
- Referential integrity enforcement differs by direction: referencing-side writes that would break it are always rejected; referenced-side deletes/updates offer a policy — RESTRICT, CASCADE, or SET NULL — chosen by what the relationship means.